LoongArch Toolchain Conventions

Loongson Technology Corporation Limited

Version 1.00

Table of Contents

Compiler Options . . . 2
- Rationale . . . 2
- Configuring Target ISA . . . 3
- Configuring Target ABI . . . 4

GNU Target Triplets and Multiarch Specifiers . . . 6

C/C++ Preprocessor Definitions . . . 8

Note: In this document, the terms "architecture", "instruction set architecture" and "ISA" are used synonymously and refer to a certain set of instructions and the set of registers they can operate upon.

Compiler Options

Rationale

Compiler options that are specific to LoongArch should denote a change in the following compiler settings:

- **Target architecture**: the allowed set of instructions and registers to be used by the compiler.
- **Target ABI type**: the data model and calling conventions.
- **Target microarchitecture**: microarchitectural features that guides compiler optimizations.

For this model, two categories of LoongArch-specific compiler options should be implemented：

- **Basic options**: select the base configuration of the compilation target. (include only `-march` `-mabi` and `-mtune`)
- **Extended options**: make incremental changes to the target configuration.

Table 1. Basic Options

Option	Possible values	Description
`-march=`	`native` `loongarch64` `la464`	Selects the target architecture, i.e. the basic set of ISA modules to be enabled.
`-mabi=`	`lp64d` `lp64f` `lp64s` `ilp32d` `ilp32f` `ilp32s`	Selects the base ABI type.
`-mtune=`	`native` `loongarch64` `la464`	Selects the type of target microarchitecture, defaults to the value of `-march`. The set of possible values should be a superset of `-march` values.

- Valid parameter values of `-march` and `-mtune` options should correspond to actual LoongArch processor implementations / families.
- In principle, different `-march` values should not imply the same set of ISA modules.

Table 2. Extended Options

Option	Possible values	Description
`-msoft-float`		Prevent the compiler from generating hardware floating-point instructions, and adjust the selected base ABI type to use soft-float calling convention. (The adjusted base ABI identifier should have suffix `s`.)
`-msingle-float`		Allow generating 32-bit floating-point instructions, and adjust the selected base ABI type to use 32-bit FP calling convention. (The adjusted base ABI identifier should have suffix `f`.)
`-mdouble-float`		Allow generating 32- and 64-bit floating-point instructions. and adjust the selected base ABI type to use 64-bit FP calling convention. (The adjusted base ABI identifier should have suffix `d`.)
`-mfpu=`	`64` `32` `0` `none` (equivalent to `0`)	Selects the allowed set of basic floating-point instructions and registers. This option should not change the FP calling convention unless it's necessary. (The implementation of this option is not mandatory. It is recommended to use `-m*-float` options in software projects and scripts.)

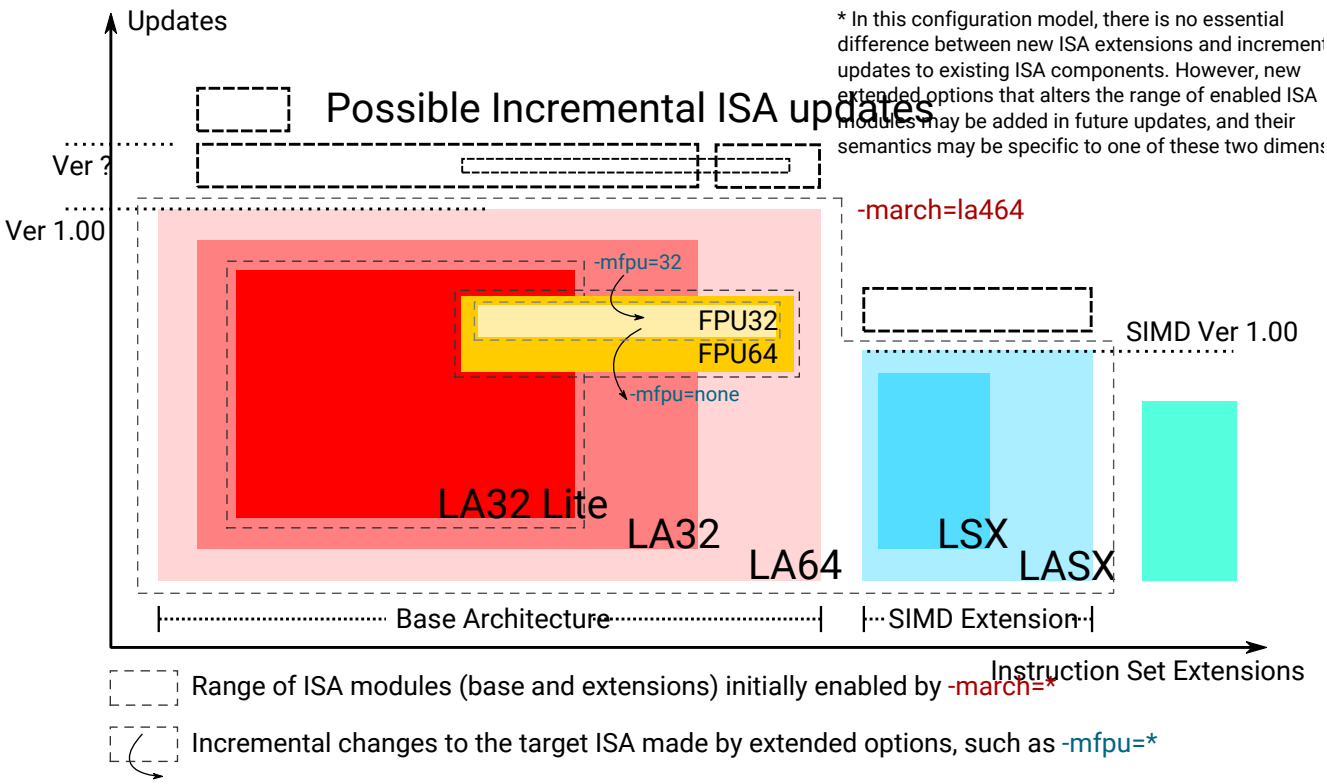
As a general rule, the effect of all LoongArch-specific compiler options that are given for one compiler invocation should be as if they are processed in the order they appear on the command line. The only exception to this rule is `-m*-float`: their configuration of floating-point instruction set and calling convention will not be changed by subsequent options other than `-m*-float`.

Configuring Target ISA

The LoongArch ISA is organized in a "base-extension" manner. For future updates, each component in the *base* or *extened* part of the ISA may evolve independently while keeping compatibility with previous versions of itself.

For this purpose, the compiler should make a modular abstraction about the target ISA. The ISA modules are divided into two categories： **base architectures** and **ISA extensions**. A **base architecture** is the core component of the target ISA, which defines the *base* set of functionalities like integer and floating-point operations, and is decided by the value of `-march`. An **ISA extension** may represent either the base of a certain *extended* ISA component or an incremental update to it, and is enabled / disabled by extended options.

For a complier, the final ISA configuration should be derived by applying various ISA extension configurations from extended options to the base ISA, which is selected via the `-march` option and consists of the base architecture and the set of default ISA extensions.



Among all ISA modules listed below, the compiler should at least implement one base architecture.

Table 3. Base Architecture

Name	`-march` values that selects this module	Description
LA64 basic architecture v1.00 (`la64v100`)	`loongarch64 la464`	ISA defined in *LoongArch Reference Manual - Volume 1: Basic Architecture* v1.00.

The following table lists all ISA extensions that should be abstracted by the compiler and the compiler

options to enable / disable them.

Table 4. ISA extensions

Name	Related compiler options	Description
Basic Floating-Point Processing Unit (`fpu*`)	`-mfpu=*` (Possible values of `*`: `none` `32` `64`)	Selects the allowed set of basic floating-point instructions and floating-point registers. This is a constituent part of the base architecture, where it gets its default value.

The following table lists the properties of all target CPU models that can serve as arguments to `-march` and `-mtune` options at the same time.

Table 5. Target CPU Models (`-march=<model>` and `-mtune=<model>`)

Name / Value	ISA modules that are enabled by default	Target of optimization
`native`	auto-detected with `cpucfg` (only on native and cross-native compilers)	auto-detected microarchitecture model / features
`loongarch64`	`la64v100` [`fpu64`]	generic LoongArch LA64 processors
`la464`	`la64v100` [`fpu64`]	LA464 processor core

Configuring Target ABI

Like configuring the target ISA, a complete ABI configuration of LoongArch consists of two parts, the **base ABI** and the **ABI extension**. The former describes the data model and calling convention in general, while the latter denotes an overall adjustment to the base ABI, which may require support from certain ISA extensions.

Please be noted that there is only ONE ABI extension slot in an ABI configuration. They do not combine with one another, and are, in principle, mutually incompatible.

A new ABI extension type will not be added to this document unless it implies certain significant performance / functional advantage that no compiler optimization techniques can provide without altering the ABI.

There are six base ABI types, whose standard names are the same as the `-mabi` values that select them. The compiler may choose to implement one or more of these base ABI types, possibly according to the range of implemented target ISA variants.

Table 6. Base ABI Types

Standard name	Data model	Bit-width of argument / return value GPRs / FPRs
`lp64d`	LP64	64 / 64
`lp64f`	LP64	64 / 32
`lp64s`	LP64	64 / (none)
`ilp32d`	ILP32	32 / 64
`ilp32f`	ILP32	32 / 32
`ilp32s`	ILP32	32 / (none)

The following table lists all ABI extension types and related compiler options. A compiler may choose to implement any subset of these extensions that contains `base`.

The default ABI extension type is `base` when referring to an ABI type with only the "base" component.

Table 7. ABI Extension Types

Name	Compiler options	Description
`base`	(none)	conforms to the LoongArch ELF psABI

The compiler should know the default ABI to use during its build time. If the ABI extension type is not explicitly configured, `base` should be used.

In principle, the target ISA configuration should not affect the decision of the target ABI. When certain ISA feature required by explicit (i.e. from the compiler's command-line arguments) ABI configuration cannot be met due constraints imposed by ISA options, the compiler should abort with an error message to complain about the conflict.

When the ABI is not fully constrained by the compiler options, the default configuration of either the base ABI or the ABI extension, whichever is missing from the command line, should be attempted. If this default ABI setting cannot be implemented by the explicitly configured target ISA, the expected behavior is **undefined** since the user is encouraged to specify which ABI to use when choosing a smaller instruction set than the default.

In this case, it is suggested that the compiler should abort with an error message, however, for user-friendliness, it may also choose to ignore the default base ABI or ABI extension and select a viable fallback ABI for the currently enabled ISA modules with caution. It is also recommended that the compiler should notify the user about the ABI change, optionally with a compiler warning. For example, passing `-mfpu=none` as the only command-line argument may cause a compiler configured with `lp64d` / `base` default ABI to automatically select `lp64s` / `base` instead.

When the target ISA configuration cannot be uniquely decided from the given compiler options, the build-time default should be consulted first. If the default ISA setting is insufficient for implementing the ABI configuration, the compiler should try enabling the missing ISA modules according to the following table, as long as they are not explicitly disabled or excluded from usage.

Table 8. Minimal architecture requirements for implementing each ABI type.

Base ABI type	ABI extension type	Minimal required ISA modules
`lp64d`	`base`	`la64v100` [`fpu64`]
`lp64f`	`base`	`la64v100` `fpu32`
`lp64s`	`base`	`la64v100` `fpunone`

GNU Target Triplets and Multiarch Specifiers

Target triplet is a core concept in the GNU build system. It describes a platform on which the code runs and mostly consists of three fields: the CPU family / model (`machine`), the vendor (`vendor`), and the operating system name (`os`).

Multiarch architecture apecifiers are essentially standard directory names where libraries are installed on a multiarch-flavored filesystem. These strings are normalized GNU target triplets. See debian documentation for details.

This document recognizes the following `machine` strings for the GNU triplets of LoongArch:

Table 9. LoongArch `machine` strings：

`machine`	Description
`loongarch64`	LA64 base architecture (implies `lp64*` ABI)
`loongarch32`	LA32 base architecture (implies `ilp32*` ABI)

As standard library directory names, the canonical multiarch architecture specifiers of LoongArch should contain information about the ABI type of the libraries that are meant to be released in the binary form and installed there.

While the integer base ABI is implied by the `machine` field, the floating-point base ABI and the ABI extension type are encoded with two string suffices (`<fabi-suffix><abiext-suffix>`) to the `os` field of the specifier, respectively.

Table 10. List of possible `<fabi-suffix>`

`<fabi-suffix>`	Description
`f64`	The base ABI uses 64-bit FPRs for parameter passing. (`lp64d` `ilp32d`)
`f32`	The base ABI uses 32-bit FPRs for parameter passing. (`lp64f` `ilp32f`)
`sf`	The base ABI uses no FPR for parameter passing. (`lp64s` `ilp32s`)

Table 11. List of possible `<abiext-suffix>`

`<abiext-suffix>`	ABI extension type
(empty string)	`base`

(Note: Since in principle, The default ISA configuration of the ABI should be used in this binary-release scenario, it is not necessary to reserve multiple multiarch specifiers for one OS / ABI combination.)

Table 12. List of LoongArch mulitarch specifiers

ABI type (Base ABI / ABI extension)	C Library	Kernel	Multiarch specifier
`lp64d` / `base`	glibc	Linux	`loongarch64-linux-gnuf64`
`lp64f` / `base`	glibc	Linux	`loongarch64-linux-gnuf32`

ABI type (Base ABI / ABI extension)	C Library	Kernel	Multiarch specifier
`lp64s` / `base`	glibc	Linux	`loongarch64-linux-gnusf`
`ilp32d` / `base`	glibc	Linux	`loongarch32-linux-gnuf64`
`ilp32f` / `base`	glibc	Linux	`loongarch32-linux-gnuf32`
`ilp32s` / `base`	glibc	Linux	`loongarch32-linux-gnusf`
`lp64d` / `base`	musl libc	Linux	`loongarch64-linux-muslf64`
`lp64f` / `base`	musl libc	Linux	`loongarch64-linux-muslf32`
`lp64s` / `base`	musl libc	Linux	`loongarch64-linux-muslsf`
`ilp32d` / `base`	musl libc	Linux	`loongarch32-linux-muslf64`
`ilp32f` / `base`	musl libc	Linux	`loongarch32-linux-muslf32`
`ilp32s` / `base`	musl libc	Linux	`loongarch32-linux-muslsf`

C/C++ Preprocessor Definitions

Table 13. LoongArch-specific C/C++ Built-in Macros：

Name	Possible Values	Description
`__loongarch__`	`1`	Defined if the target is LoongArch.
`__loongarch_grlen`	`64`	Bit-width of general purpose registers.
`__loongarch_frlen`	`0` `32` `64`	Bit-width of floating-point registers (`0` if there is no FPU).
`_LOONGARCH_ARCH`	`"loongarch64"` `"la464"`	Processor model as specified by `-march`. If `-march` is not present, the build-time default should be used.
`_LOONGARCH_TUNE`	`"loongarch64"` `"la464"`	Processor model as specified by `-mtune`. If `-mtune` is not present, the build-time default should be used.
`__loongarch_lp64`	(none)	Defined if ABI uses the LP64 data model and 64-bit GPRs for parameter passing.
`__loongarch_hard_float`	(none)	Defined if floating-point/extended ABI type is `single` or `double`.
`__loongarch_soft_float`	(none)	Defined if floating-point/extended ABI type is `soft`.
`__loongarch_single_float`	(none)	Defined if floating-point/extended ABI type is `single`.
`__loongarch_double_float`	(none)	Defined if floating-point/extended ABI type is `double`.
`_LOONGARCH_SZINT`	(omitted)	Bit-width of C/C++ `int` type.
`_LOONGARCH_SZLONG`	(omitted)	Bit-width of C/C++ `long int` type.
`_LOONGARCH_SZPTR`	(omitted)	Bit-width of C/C++ pointer types.